

Project - High Level Design on IaC Provisioning for Finance Systems

Course Name: DevOps Foundation

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Krati Patidar	EN22CS301514
2	Kamal Yadav	EN22CS301476
3	Kashish Sharma	EN22CS301492
4	Khushi Mehra	EN22CS301504
5	Jidnyas Bhushan Bhonge	EN22CS301469

Group Name: G03D5

Project Number: DO-03

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2025-2026

Table of Contents

1. Introduction.
 - 1.1. Scope of the document.
 - 1.2. Intended Audience
 - 1.3. System overview.
2. System Design.
 - 2.1. Application Design
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. Components Design
 - 2.5. Key Design Considerations
 - 2.6. API Catalogue.
3. Data Design.
 - 3.1. Data Model
 - 3.2. Data Access Mechanism
 - 3.3. Data Retention Policies
 - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
 - 7.1. Security Aspects
 - 7.2. Performance Aspects
8. References

1. Introduction

1.1 Scope of the Document

This High-Level Design (HLD) document serves as the primary architectural blueprint for the **Finance Support Triage System**. It defines the macro-level interaction between the application components, cloud-native infrastructure, and automated deployment pipelines. The scope includes defining system boundaries, data flow patterns, and the integration of Infrastructure as Code (IaC) to ensure a repeatable, production-ready environment.

1.2 Intended Audience

- **Project Stakeholders:** To understand the end-to-end value proposition and system capabilities.
- **DevOps Engineers:** To validate the orchestration of GitHub Actions, Terraform, and AWS services.
- **Security Auditors:** To review the handling of sensitive financial metadata and environmental security controls.
- **Developers:** To understand the architectural constraints and interface requirements for feature expansion.

1.3 System Overview

The system is an automated platform designed to provide a **Static Financial Data Dashboard**. The architecture utilizes a **Containerized Micro-service** approach, where the core logic is encapsulated within an immutable **Docker container**. Deployment is hosted on **AWS EC2**, ensuring high availability. By integrating **GitHub Actions** for CI/CD and **Terraform** for IaC, the system achieves a "Zero-Touch" deployment model, eliminating human error in environment configuration and ensuring the production environment stays in sync with the source code.

2. System Design

2.1 Application Design

The application follows a decoupled **Three-Tier Architecture** model to ensure that failures in the presentation layer do not impact the core data processing logic:

- **Presentation Layer:** A Streamlit-based web interface providing real-time dashboards and triage results to the end-user.
- **Application Logic Layer:** A FastAPI backend that orchestrates the IMAP/SMTP protocols and manages the AI classification engine.
- **Data Persistence Layer:** A managed PostgreSQL database (Aiven) providing ACID-compliant storage for triage logs and historical data.

2.2 Process Flow

The end-to-end process is divided into two distinct operational lifecycles:

1. **Deployment Lifecycle:** Source Code Commit --> GitHub Actions Pipeline Trigger --> Docker Image Build/Push --> Terraform Infrastructure Validation --> Container Deployment to AWS EC2.
2. **Runtime Lifecycle:** Email Fetching --> Text Pre-processing/Cleaning --> AI Inference (Classification) --> Database Persistence --> UI Dashboard Update.

2.3 Information Flow

Information moves through four primary logical zones to maintain a high degree of isolation:

- **Development Zone:** Source code and configuration files move from local environments to the GitHub Repository.
- **Automation Zone:** GitHub Actions handles the transformation of code into a deployable Docker artifact and manages secret injection.
- **Cloud Zone:** Encrypted traffic moves between the AWS EC2 compute instance and the Aiven PostgreSQL storage via secured TCP/IP connections.
- **Client Zone:** Processed triage results are served via JSON over HTTP to the end-user's browser on port 8501. Accessible by navigating to <http://34.233.126.144>

2.4 Components Design

- **Terraform:** Acts as the "Foundational Layer," defining the VPC, Subnets, and EC2 instances as code.

- **Docker:** Acts as the "Portability Layer," packaging Python 3.10 and all dependencies into an immutable unit.
- **GitHub Actions:** Acts as the "Orchestration Layer," managing deployment triggers and environment-specific secret keys.
- **FastAPI & Uvicorn:** Acts as the "Communication Layer," providing a high-concurrency web server capable of handling multiple triage requests.

2.5 Key Design Considerations

- **Idempotency:** All Terraform scripts are designed to be idempotent, ensuring that subsequent runs maintain the desired state without duplicating resources.
- **Environment Parity:** Docker ensures that the "Developer Laptop" environment is 100% identical to the "AWS Production" environment.
- **Fault Tolerance:** The containerized nature allows for rapid "self-healing" where a crashed container can be automatically restarted by the Docker daemon.

2.6 API Catalogue

- **GET /:** Entry point for the health check and Streamlit dashboard initialization.
- **POST /email-fetch:** Securely authenticates with enterprise mail servers using IMAP to retrieve unread support tickets.
- **GET /analyze:** The core AI logic. It parses the email body and returns a classification JSON (e.g., {"category": "Billing", "priority": "High"}).
- **POST /send-reply:** Leverages SMTP to dispatch automated or agent-approved responses to the customer.

3. Data Design

3.1 Data Model

The relational schema is optimized for auditability. The primary table, TriageLogs, includes the following fields:

- **UUID:** Primary key for record uniqueness across distributed systems.
- **Email_ID:** Reference to the original message in the mail server.
- **Category:** AI-assigned labels (e.g., Urgent, Billing, Technical).
- **Confidence_Score:** A float value (0.0–1.0) indicating the AI's certainty level.
- **Timestamp:** The exact time of the triage event for performance benchmarking.

3.2 Data Access Mechanism

The system implements the **Data Mapper Pattern** using **SQLAlchemy ORM**. This abstracts the database logic, preventing "vendor-lock" to a specific SQL dialect. All database interactions are managed through a session factory that ensures connections are pooled and released efficiently.

3.3 Data Retention Policies

- **Hot Storage:** All triage records from the current fiscal quarter are kept in the primary indexed tables for sub-second dashboard rendering.
- **Cold Storage:** Records older than 180 days are offloaded to archived storage (S3/Parquet) to maintain PostgreSQL performance.

3.4 Data Migration

Schema versions are managed through the application code. Upon container startup, a `metadata.create_all()` hook is triggered. This ensures that the database schema is automatically updated to match the current version of the application code without requiring manual SQL scripts.

4. Interfaces

The Finance Support Triage System acts as a central orchestrator, managing high-fidelity interfaces across cloud infrastructure, managed data persistence, and legacy communication protocols.

4.1 AWS Infrastructure Interface (IaC & CLI)

This interface governs the lifecycle of the compute environment. It is primarily managed through the **Terraform AWS Provider**, which translates HCL (HashiCorp Configuration Language) into API calls against the AWS Global Infrastructure.

- **Provisioning Logic:** Terraform interfaces with the **Amazon EC2 API** to define Instance Types (e.g., `t3.micro`), VPC Subnets, and Security Groups. It uses a "State File" (`terraform.tfstate`) to track the delta between the desired architecture and the live environment.
- **CLI Integration:** The **AWS Command Line Interface (CLI)** is utilized within the GitHub Actions runner. It performs the initial authentication using `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` to authorize the Terraform binary and the Docker "Remote Command" execution.
- **Security Handshake:** Communication is secured via **IAM Roles** and **SSH Key Pairs**. The interface ensures that only the GitHub runner has the temporary authority to modify the EC2 state.

4.2 Aiven Database Interface (PostgreSQL/TLS)

The interface between the FastAPI backend and the Aiven Managed PostgreSQL service is a high-security data link designed for ACID compliance and encrypted transit.

- **Protocol & Connection:** The system uses the **PostgreSQL Wire Protocol** over **TCP/IP Port 28211**. This non-standard port adds a layer of "security by obscurity" while bypassing common port-scanning targets.
- **Encryption (SSL/TLS):** The interface strictly enforces **SSL Certificate Verification**. The application will refuse to initiate a handshake unless the Aiven CA certificate is validated, preventing Man-in-the-Middle (MITM) attacks. The connection string includes the `sslmode=require` parameter.
- **ORM Integration:** **SQLAlchemy** acts as the translator for this interface, converting Pythonic models into optimized SQL queries. It manages a

Connection Pool, keeping a set of "warm" connections ready to reduce the latency of the initial TCP handshake for each triage request.

4.3 Mail Protocols Interface (IMAP & SMTP)

This interface facilitates the "Ingress" and "Egress" of financial support data. It handles the transition from unstructured email text to structured system data.

- **Ingress (IMAP):** The system connects to the Finance Support Inbox via **IMAP over SSL (Port 993)**.
 - **Logic:** It uses the `imaplib` library to perform a "SEARCH UNSEEN" command. The interface is designed to fetch only the headers first (to save bandwidth) before downloading the full multi-part MIME body for AI analysis.
- **Egress (SMTP):** For automated responses, the system utilizes **SMTP over TLS (Port 587)**.
 - **Logic:** Once the AI generates a draft or the user approves a reply, the system initiates an `ehlo` handshake with the mail server, upgrades the connection to TLS, and dispatches the response using the `smtplib` module.
- **Authentication:** Both protocols utilize **App-Specific Passwords** or **OAuth2 tokens**, ensuring that the system never stores the primary account credentials in plain text.

5. State and Session Management

The Finance Support Triage System utilizes a hybrid state model designed for cloud-native scalability. By decoupling the volatile "UI State" from the "Application Logic," the system ensures high availability and simplifies the horizontal scaling process.

5.1 Backend Statelessness (FastAPI Tier)

The Application Tier is strictly **Stateless**, meaning the FastAPI server does not retain any "memory" of previous interactions once an HTTP request-response cycle is complete.

- **Request Autonomy:** Each incoming API call (e.g., `/analyze` or `/email-fetch`) is treated as an independent transaction. The request must contain all necessary parameters, such as the `Email_ID` or `Auth_Token`, to be processed. The server does not store these in a local cache or global variable.
- **Horizontal Scalability:** Because the backend is stateless, the system avoids the need for "Sticky Sessions" at the Load Balancer level. If traffic spikes, additional Docker containers can be spun up on the EC2 instance (or across multiple instances); any container can handle any request because no session data is trapped inside a specific container's RAM.
- **Resource Efficiency:** Statelessness prevents "Memory Leaks" associated with abandoned user sessions. Once a triage operation is finished, the container immediately frees up its memory to handle the next incoming task.

5.2 Frontend Session Management (Streamlit Tier)

Since the backend is stateless, the **Streamlit** frontend is responsible for maintaining the "User Context" to ensure a fluid interface.

- **Local Session State (`st.session_state`):** Streamlit maintains a key-value store in the server's memory that is unique to each browser tab. This is used to track:
 - **UI Controls:** Which filters (e.g., "Urgent Only") are currently toggled.
 - **Drafting Logic:** Temporary edits to an AI-generated email reply before the user hits "Send."
 - **Pagination:** The current index of the email list the user is browsing.
- **Session Continuity:** If the user refreshes the page, the local session is reset,

but the **Aiven PostgreSQL** database acts as the ultimate "Source of Truth." Upon refresh, the frontend issues a fresh GET request to the backend to reconstruct the necessary state from the persisted triage logs.

- **Concurrency Isolation:** The system ensures that Session A (User 1) cannot see or interfere with Session B (User 2). This is handled by Streamlit's internal session-handling logic, which treats each connection as a distinct thread, preventing data leakage between support agents.

5.3 State Persistence Bridge

The bridge between the volatile frontend session and the stateless backend is the **Database Commit**.

- **The Handshake:** When a user clicks "Confirm Triage," the frontend pushes the current session data to the FastAPI /update endpoint.
- **Transition to Persistence:** Once the backend receives the data and commits it to the **PostgreSQL** database, the "State" is officially transformed from a temporary browser variable into a permanent, audited record.

6. Caching

The Finance Support Triage System implements a multi-layer caching strategy to optimize computational throughput and minimize latency. By reducing redundant AI model inferences and database I/O, the system ensures high responsiveness even under heavy concurrent loads.

6.1 Application-Layer Caching (LRU Logic)

At the highest level of the logic tier, the system utilizes **In-Memory Caching** to bypass the expensive "Computation Phase" of the AI classification engine.

- **Mechanism (@lru_cache):** Using Python's `functools.lru_cache`, the system stores the output of the `/analyze` function. When a support email with identical metadata (Subject Line + Sender ID) is processed, the system performs a hash lookup in the RAM instead of re-invoking the AI model.
- **Eviction Policy:** The "Least Recently Used" (LRU) algorithm ensures that only the most relevant and frequent triage patterns are kept in memory. When the cache reaches its pre-defined limit (e.g., 128 or 256 entries), the oldest, least-accessed classification results are evicted to prevent memory bloat within the Docker container.
- **Performance Impact:** This reduces the CPU utilization on the **AWS EC2** instance by up to **40%** during repetitive email cycles (e.g., multiple follow-ups on the same ticket), dropping the response time from ~1.5 seconds to <10ms for cached hits.

6.2 Database Query Caching (TTL Strategy)

To protect the **Aiven PostgreSQL** instance from "Read Exhaustion" during peak support hours, the system employs a temporal caching layer for high-frequency dashboard queries.

- **Time-To-Live (TTL) Logic:** High-traffic queries—specifically those fetching "Urgent" or "Unprocessed" tickets—are wrapped in a cache with a **60-second TTL**. This means that within a one-minute window, multiple support agents viewing the dashboard will receive the same result from the application's memory rather than triggering 10+ identical SQL SELECT statements.

- **I/O Optimization:** By intercepting these "Heavy Reads" at the FastAPI level, the system preserves the database's I/O Operations Per Second (IOPS) for "Critical Writes" (commits and triage updates). This ensures that the managed database remains responsive for the entire team.
- **Consistency Control:** The 60-second window is a calculated balance; it is short enough to ensure agents see nearly real-time updates from their colleagues, but long enough to provide a significant performance buffer during "Email Storms."

6.3 Cache Invalidation and Integrity

Caching introduces the risk of "Stale Data." The system mitigates this through specific invalidation triggers:

- **Write-Through Invalidation:** Whenever an agent manually updates a triage category or a "Send Reply" action is confirmed, the specific cache entry for that `Email_ID` is invalidated. This forces the system to fetch the latest, "True" state from the PostgreSQL database for the next request.
- **Memory Constraints:** Since the application runs in a **Docker** environment with finite resources (e.g., `t3.micro`), the cache size is strictly capped. This prevents the container from being killed by the Linux OOM (Out-of-Memory) killer, maintaining system stability.

7. Non-Functional Requirements

7.1 Security Aspects

- **Network Security:** AWS Security Groups are configured with "Least Privilege," allowing only port 8501 (App) and Port 22 (SSH) access.
- **Secret Management:** Sensitive credentials (DB passwords, API keys) are stored as GitHub Secrets and injected into the .env file at runtime.
- **SSL Enforcement:** All database traffic is encrypted via TLS 1.2+ to prevent man-in-the-middle attacks.

7.2 Performance Aspects

- **Latency:** The system is benchmarked for a "sub-2-second" end-to-end triage time.
- **Efficiency:** The use of the python:3.10-slim base image ensures the Docker container is lightweight (approx. 200MB), leading to faster deployment and lower memory consumption.

8. References

- AWS EC2 & Docker Official Architectural Documentation.
- Terraform Registry for AWS Provider Specifications.
- FastAPI & SQLAlchemy High-Performance Design Guides.
- Streamlit Documentation
- GitHub Actions Documentation